

Intrusion Detection Using Convolutional Neural Network: A Color Mapping Approach on NSL-KDD Dataset

Md. Abrar Faiaz
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh
abrarfaiaz20@gmail.com

Dipankar Mitra
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh
dipankar.mitra4854@gmail.com

Ranat Das Prangon
Bangladesh University of Engineering
and Technology
Dhaka, Bangladesh
ranatdasprangon@gmail.com

Abstract

Converting any kind of data to image data can make the dataset suitable for Convolutional Neural Networks (CNNs). In this study, the NSL-KDD dataset was converted to image data using the color mapping technique and using CNN a good accuracy of 98.91% and aggregated f1 score of 0.91 was achieved. Here an image representation of each row was generated using both Hue-Saturation-Value (HSV) and Viridis colormap. Though some attack types were misclassified by the model, no attack sample of the validation dataset was classified as normal. For model training, five CNN architectures were evaluated by transfer learning from their pre-trained weights. It was found that ResNet18 performs best among all the five architectures evaluated. ResNet18 uses 1x1 convolution to reduce the number of parameters used. It means that for the classification of the colormap of this dataset complex CNN architectures are not necessary. Fine-tuning of the ResNet18, the best-performing architecture, was done using 50 epochs using an optimal learning rate. However, the accuracy mentioned above was found only after 4 epochs demonstrating good efficiency of the model training. The NSL-KDD dataset contains information about network intrusion in tabular format. Hence, this model can be used for intrusion detection purposes.

CCS Concepts

• Security and Privacy; • Intrusion/anomaly detection and malware mitigation; • Intrusion detection systems;

Keywords

IDS, Intrusion Detection, NSL-KDD, Machine Learning, Resnet, Color mapping

ACM Reference Format:

Md. Abrar Faiaz, Dipankar Mitra, and Ranat Das Prangon. 2024. Intrusion Detection Using Convolutional Neural Network: A Color Mapping Approach on NSL-KDD Dataset. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3704522.3704541>



This work is licensed under a Creative Commons Attribution International 4.0 License.

NSysS '24, December 19–21, 2024, Khulna, Bangladesh
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1158-9/24/12
<https://doi.org/10.1145/3704522.3704541>

1 INTRODUCTION

Nowadays cybersecurity is a major concern in all sectors of the contemporary digital landscape, and cybersecurity has emerged as a paramount concern across all sectors [8, 10]. Proper and initial detection of any malicious activities is very important for ensuring the security of confidential data and the integrity of the system. Typically, a cyber-attack starts with breaching the restricted part of a network system or a server bypassing its security mechanisms. Such breaches compromise the Confidentiality, Integrity, and Availability (CIA) of a system, collectively known as intrusions [5].

To prevent intrusion from the network system, various kinds of Intrusion Detection Systems (IDS) were developed as shown in 1. IDS is a combination of hardware and software components to detect suspicious attempts on the network [32]. Broadly, intrusion detection can be categorized into three main types[21],

- Signature-based Detection (SD).
- Anomaly-based Detection (AD).
- Stateful Protocol Analysis (SPA).

The signature-based approach, often referred to as the anti-virus method, works by matching incoming data to pre-existing patterns of known attacks. While effective at identifying previously recognized threats, it falls short in predicting novel or unknown attacks. Conversely, stateful protocol analysis seeks to identify unexpected sequences of commands, but its application is resource intensive. This is where anomaly-based detection gains prominence. By analysing behavioural patterns, it utilizes statistical methods and connection data to flag suspicious network activities. Although it may occasionally generate false positives, this approach excels in detecting previously unseen threats.

For this anomaly-based detection system, a model needs to be built depending on various data. Numerous datasets were made depending on previous attacks such as AATCT-IDS [23], LSPR23 [7], CSE-CIC-IDS-2018 [29], KDD CUP 99 [3], NSL-KDD [42], and so on. Among these, the NSL-KDD dataset stands out for its detailed composition, comprising 41 attributes such as connection time, network protocol, login status, the number of failed login attempts, root shell usage, and more. To make IDS from this kind of dataset, various statistical approaches have been taken [14, 31, 40]. Also, some approaches use traditional Machine learning (ML). However traditional ML models cannot utilize the power of big datasets where a huge number of numerical and categorical variables get involved [9, 16, 26, 30].

To solve this issue, various Deep Learning (DL) approaches were taken previously [2, 12, 15, 22, 25, 38]. These models can detect intrusion with higher efficiency and higher accuracy. DL has a more robust training technique than traditional ML algorithms. DL

Table 1: Various studies on Intrusion Detection System

Dataset Used	Method Used	Accuracy (%)	Ref
CICIDS2017, ECU-IoHT, WUSTL-EHMS-2020	Machine learning models	93.6 (best)	[39]
KDD '99	Backpropagation, Unsupervised learning	91.26	[37]
XCANIDS	Dynamic graph	99.94	[31]
-	CNN,	93.25	[20]
	LDP-ecGAN,	94.10	
	DFD Collab,	96.36	
	Pb-fdGAN,	98.16	
UAV Attack	E-DIDS	97.8	[35]
NBaaS, CICIDS-2017, and ToN-IoT	LSTM,	99.96 (NBaaS),	[6]
	GRU,	99.97 (CICIDS-2017,	
	Bi-LSTM,	99.88 (ToN-IoT)	
	Modified Bi-LSTM		

can nearly learn anything, but it needs a bunch amount of data compared to the traditional ML models. However, the transfer learning method from pre-trained DL models on similar kinds of data can be used with a comparatively small amount of data [24]. The NSL-KDD dataset contains more than 125,000 rows [42] which is enough for transfer learning. So, a DL approach can be taken for this dataset.

As one of the most descriptive datasets, NSL-KDD contains 41 attributes such as connection time, network protocol, login status, no of failed login attempts, root shell used or not, and so on. It classifies attacks into 39 classes. [4]. Several ML and DL models were built previously to detect intrusion from the NSL-KDD dataset [28, 36, 41]. A study by S. Alrayes et al. used a Coevolutionary Neural Network (CNN) and got 99.728% accuracy [41]. That model merged 36 attack classes of the dataset into 4 broad classes. Hence, that model was only able to categorize intrusion only in those 4 categories. However, in the current study, a model has been built that will be able to detect and categorize intrusion into 12 smaller classes for a more precious decision.

Initially, each row of the NSL-KDD dataset was transformed into an imagery that represents the row. Then the Coevolutionary Neural Network (CNN) was used to train the model here.

2 DATASET

The NSL-KDD dataset is a refined version of the original KDD '99 dataset [34]. In this dataset, duplicate records are omitted, which makes the ML models less vulnerable to redundant classes. The dataset contains a total of 42 attributes and 41 of them are used as features. These features are categorized into four primary types: Basic (B), Content (C), Time Traffic (T), and Host Traffic (H) [1]. The dataset contains 32 numerical and 9 categorical variables, as outlined in 2. The NSL-KDD dataset is particularly well-suited for the color mapping approach due to its rich and diverse set of features. It holds a wide range of values that can be strongly encoded into RGB color representations. This allows the CNN-based

model to capitalize on its strength in image pattern recognition in order to detect subtle differences in intrusion patterns.

2.1 Basic type data:

The basic features provide fundamental information about the network connection. These attributes include parameters such as connection duration, protocol type, network service type, the volume of data bytes transferred, and urgency of the connection, among others. This information gives an overview of a connection [4]. However, relying solely on these basic features makes it challenging to differentiate between normal connections and intrusion attempts due to their general nature.

2.2 Content type data

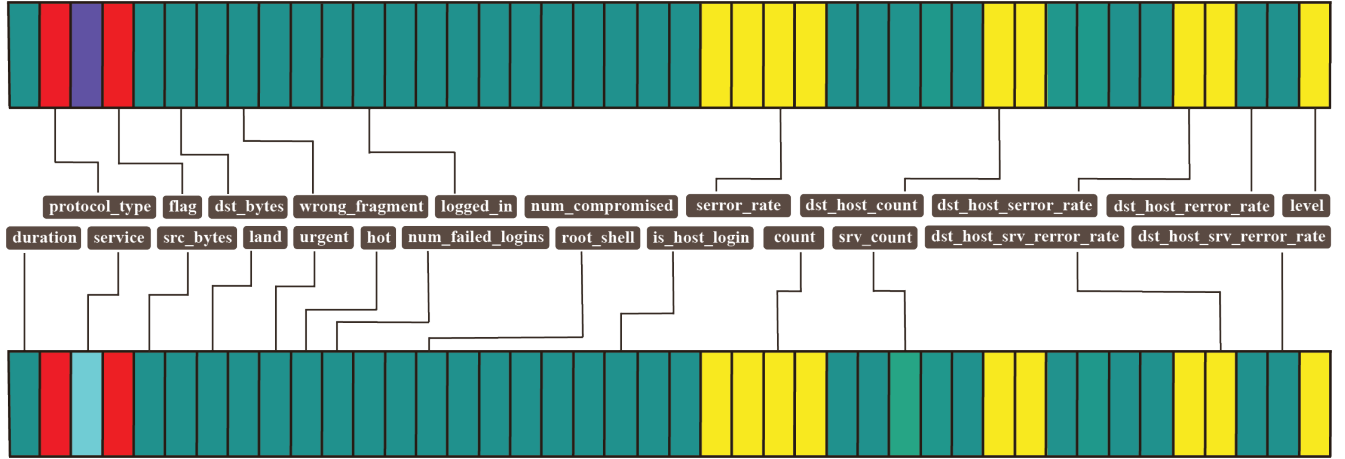
This contains information about what types of content are accessed in the connection. The variable here is whether the connection entering a sensitive system directory or executing program, is_logged_in, Num_failed_logins, is_using_root_shell, su_attempted, num_shell using, etc [4]. This information gives an idea about what type of connection is it. However, most of the time intrusions begin with accessing sensitive directories or executing a program with superuser access. Thus, this information is most important for an Intrusion Detection System (IDS).

2.3 Time Traffic and Host Traffic

Unlike the previous categories, which focus on individual connections, Time Traffic and Host Traffic features provide insight into the broader network traffic at the time of the connection. These variables include the number of connections at that moment, the number of connections to the same port, the error rate of connections, and the number of connections sharing the same destination IP address [4]. This data helps to compare the information of a single connection to the overall state of other connections. Thus, this information plays a very important role in detecting intrusions in the system.

Table 2: Different kinds of variable counts in the NSL-KDD dataset

Variable Types	Nos. Categorical	Nos. Numerical	Total Variables
Basic	4	5	9
Content	5	8	13
Time Traffic	0	9	9
Host Traffic	0	10	10

**Figure 1: Color map of a single row of the NSL-KDD dataset. Each color strip of the colormap represents a single cell of a row.**

3 METHODOLOGY

3.1 Data Preprocessing and converting to image

3.1.1 Class Selection. Initially, the number of attack classes and entries per class was determined. The class with very low entries (less than 50) was omitted as the model may not be able to learn from those entries. Also, there was a problem regarding cross-validation for those classes. Thus 12 attack classes were selected as significant classes which are back, ipsweep, Neptune, nmap, normal, pod, portsweep, satan, smurf, teardrop, warezclient, and other classes were merged to a class named 'others.' After that preprocessing was applied to the NSL-KDD dataset. After selecting the data the numerical columns were scaled using a standard scaler and the categorical columns were encoded using label encoding

3.1.2 Image Generation with color mapping. To transform tabular data into image representations, a color-mapping approach was utilized, wherein each row in the dataset was converted into a corresponding image as shown in Figure 2. Columns were categorized as either numerical or categorical based on data types. For categorical variables, each unique value was encoded and assigned a distinct RGB color using the Hue, Saturation, and Value (HSV) colormap. The HSV colormap uses 360° hue spectrum for color mapping. It is ideal for categorical variable because for its circular behavior it doesn't have any order dependency which is necessary for a categorical feature.

For numerical variables, values were normalized between a pre-defined range $[-1,1]$ using normal scaller, and mapped to RGB

colors using the Viridis colormap. This colormap has a smooth transition from dark to bright, which can represent numerical range and low to high values. Here each color represents a value.

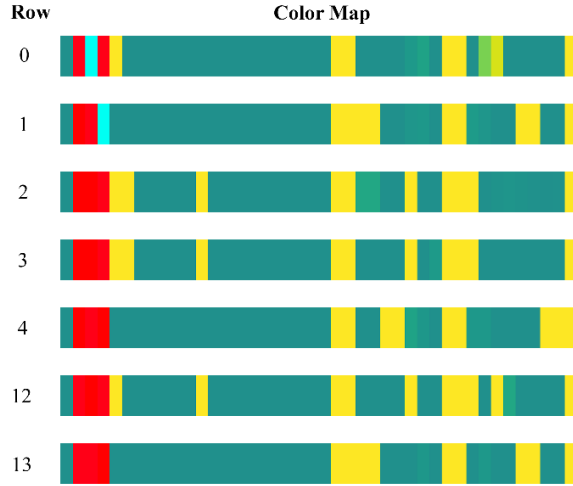
Each row of the dataset was visualized as a small horizontal image of fixed dimensions, with columns represented as equally spaced color strips as shown in 1. Generated images were saved with labels from the target variable. These images were further used for image classification.

3.2 Model Selection and Training

FastAI's DataBlock API was used to define the data pipeline for training the model. The API was used to specify that the inputs are images via ImageBlock and the outputs are attack categories via CategoryBlock. The RandomSplitter was applied to split the dataset into training and validation sets, with 20% reserved for validation. Pretrained architectures such as ResNet18, ResNet15, ResNet101, BEiT2_base_patch16_224, and ResNet 152 were evaluated by transfer learning to find the best-performing architecture. Some details of the ResNet architectures are given in 3. By using transfer learning, the training process was accelerated while ensuring that the model could efficiently learn from the available data. The model was initialized with these pre-trained weights, and a DataLoaders object was created to handle the batching and transformations of the data during training. This setup provided a robust framework for training on the heatmaps while optimizing performance and accuracy.

Table 3: Details of ResNet architectures

Architecture	Layer count	Parameter count	Reference
ResNet18	18	12 million	[11]
ResNet50	50	25 million	[11, 13]
ResNet101	101	45 million	[11]
ResNet152	152	60 million	[11, 13]

**Figure 2: Color map of some rows of the dataset**

3.3 Fine-tuning of learning rate

The learning rate finder of FastAI was used to find the optimal learning rate. The pre-trained weights of the model were modified to the NSL-KDD dataset over the course of 50 epochs after the optimal learning rate was established. To ensure that the model learns the specific properties relevant to intrusion detection, it is fine-tuned using the generalized information included in the pre-trained model. From the 50 epochs, the epoch for the lowest validation loss was selected to avoid overfitting the data. The approach maximizes the model's ability to classify the NSL-KDD dataset's 12 attack classes accurately, achieving high efficiency in detecting a wide range of network intrusions. After model training its efficiency was evaluated by generating a confusion matrix using all the 12 attack classes.

3.4 Analyzing evaluation metrics

Lastly, the confusion matrix was prepared.

And, from confusion matrix components precision, recall and f1 score for each class was evaluated and finally macro precision, macro recall and macro f1 score was calculated for the final model.

3.4.1 Confusion matrix components.

- True Positives (TP) means correctly classifying an entry to a class
- True negative (TN) for each class refer to the correctly classifying that that entry doesn't belong to that class.
- False positive (FP) means incorrectly classifying an entry as belonging to a class when it doesn't belong to that class.
- False Negative (FN) means incorrectly classifying an entry as not belonging to a class when it belongs to that class.
- As it is a multiclass classification TP, TN, FP and FN was calculated for each class.

3.4.2 Precision. Precision means no of instances which are predicted correctly form all of the instances those are predicted as that class. The sum of TP and FP for a class is the number of predicted instances for that class. Hence, precision can be calculated from the equation below.

$$\text{Precision} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Positive (FP)}}$$

3.4.3 Recall. This means the proportion of correctly predicted instances for a class out of all actual instances of that class. The sum of TP and FN for a class is the number of actual instances for that class. Thus, recall can be calculated from equation below.

$$\text{Recall} = \frac{\text{True Positive (TP)}}{\text{True Positive (TP)} + \text{False Negative (FN)}}$$

3.4.4 F1 score. It is the harmonic mean of precision and recall for a class, balancing both metrics.

$$\text{F1 score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

3.4.5 Aggregated average. As for multiclass classification the metrics is calculated for each individual class and aggregated average need to be calculated for easy interpretation of the model performance. Thus aggregated average of each metrics was calculated.

$$\text{Aggregated Average} = \frac{1}{C} \sum_{i=1}^C \text{Metrics}_i$$

Where,

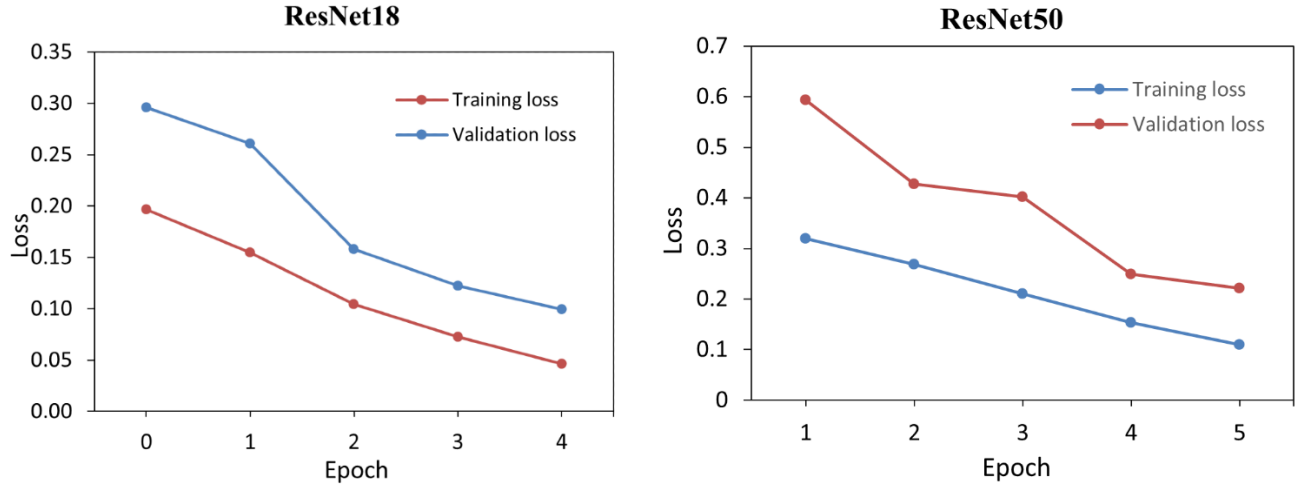
C = Number of classes

Metrics_i = Value of precision, recall or f1 score of a class

Aggregate precision, aggregated recall and aggregated f1 score was calculated from this equation.

Table 4: Performance of Various Deep Learning Architectures

Model architecture	Best training loss	Best validation loss	Best accuracy	Average Training Times (sec)
ResNet18	0.0466	0.0499	0.9891	3.0
ResNet50	0.1101	0.0961	0.9813	4.4
BEiT _{v2} _base_patch16_224	1.0111	0.9447	0.7461	25.8
ResNet101	0.2471	0.0923	0.9720	7.3
ResNet152	0.2602	0.7951	0.9564	10.3

**Figure 3: Training and Validation loss of ResNet18 and ResNet50 architecture over epochs**

4 RESULTS AND DISCUSSION

In this part, the results of the analysis of various models are shown. It includes the performance of various architectures and thus the hyperparameter tuning results of the best-performing architecture.

4.1 Performance of various architectures

In this study, the machine learning architectures deployed are ResNet18, ResNet50, ResNet101, ResNet152, and BEiT_{v2}_base_patch16_224, all leveraging pre-trained architectures. The ResNet 18 performed best among all the models. The data of these training models are shown in 4. The details of those models are given below.

4.1.1 ResNet18. ResNet18 displayed superior performance in comparison with the other models, during the five epochs, the model continuously decreased both training and validation losses; accordingly, the training loss went from 0.1967 to 0.0466, and the validation loss came down to 0.0527 (3). The corresponding accuracy increased and reached the maximum value of 98.75% after five epochs. Also, its average training time was lower (3 sec) due to less complexity in the model architecture. ResNet (Residual Network) is a deep convolutional neural network architecture designed to address the vanishing gradient problem, which can hinder training in deep networks [11]. ResNet18, in particular, has 18 layers and is built upon residual blocks. Each block includes shortcut connections that bypass one or more layers, enabling efficient gradient flow and

feature extraction [27]. This architecture balances depth and simplicity, making it effective for datasets like NSL-KDD. ResNet18's relatively shallow structure compared to deeper architectures like ResNet50 and ResNet101 helped it achieve superior performance on the transformed color-mapped images, with reduced risk of overfitting and lower computational requirements. This balance allowed it to generalize effectively across various intrusion classes while maintaining efficient training times.

4.1.2 ResNet50. ResNet50 architecture also showed quite good accuracy but showed more variation in its validation loss compared to the ResNet18. Training losses were recorded over a period of five epochs. The model achieved a peak accuracy of 98.13%, but its training and test validation were higher compared to ResNet18 (4). Its training time was also higher compared to the ResNet18 model. This is due to the higher complexity of its model architecture [17].

4.1.3 BEiT_{v2} base_patch16_224. The accuracy score for BEiT_{v2}_base_patch16_224 was notably poor on this dataset. Though its performance improved over epochs, even after the ninth epoch, its accuracy was 74.61%. Which is very low compared to other ResNet models. Despite ongoing training, the validation losses remained large, 1.0111 for the train set and 0.9447 for the validation set. The train loss and validation loss over epochs are shown in Figure 4. Its training time was notably higher, 25.8 seconds than all other models as shown in 4. So, it is no good choice for the current system.

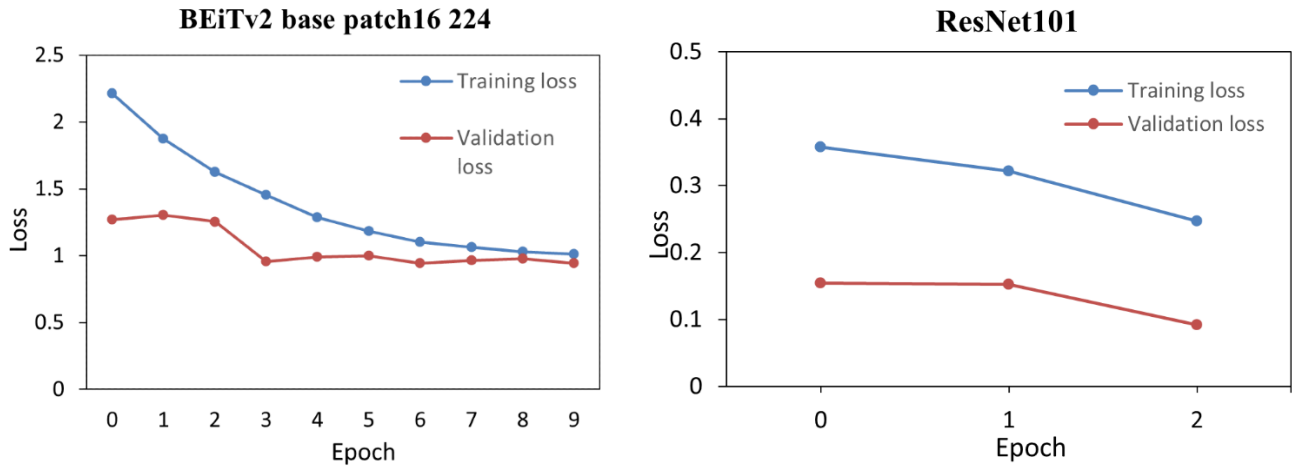


Figure 4: Training and Validation loss of BEiTv2 base patch16 224 and ResNet101 architecture over epochs

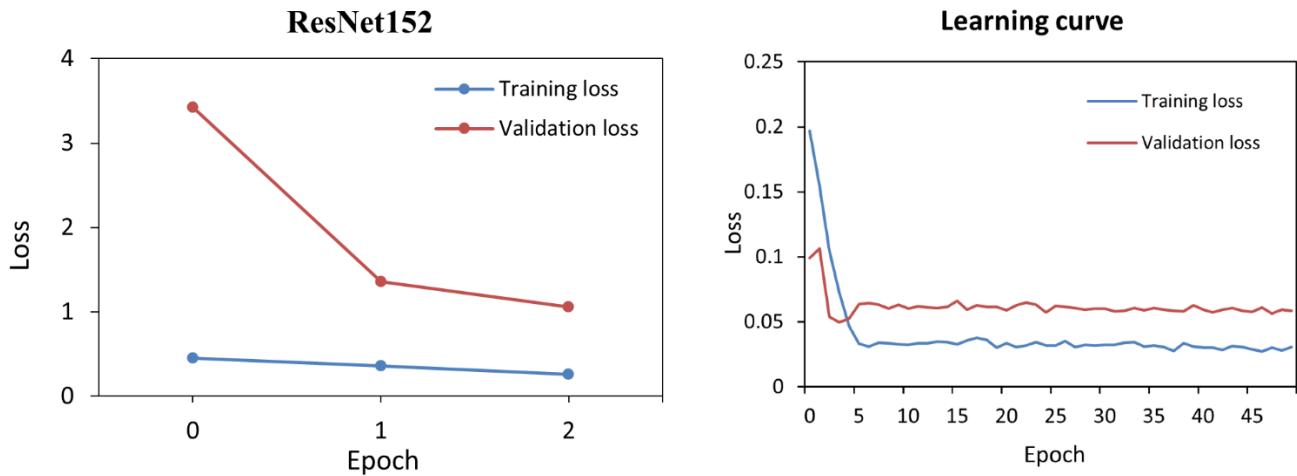


Figure 5: Training and Validation loss of ResNet152 architecture (left) and ResNet18 after fine tuning of learning rate

4.1.4 ResNet101. This model had higher accuracy than BEiTv2_base_patch16_224, but its accuracy remained lower than that of ResNet18 and ResNet50. Its accuracy was 97.20% while ResNet18 had 98.91% as shown in 4. Also, its training time was higher than ResNet18 and ResNet50. This is because ResNet101 has 101 layers and ResNet18 and ResNet50 have 18 and 50 layers correspondingly as shown in 3. This figures out the higher complexity of the ResNet101 model, resulting in a higher training time. The training and validation losses are shown in 4.

4.1.5 ResNet152. The ResNet152 is a more complex model than previously discussed ResNet models. Hence, its training time is higher than all of the models discussed above. However, its accuracy was 95.64% which is the worst of all the ResNet models evaluated here. Also, its test and validation losses were higher than ResNet18. So, after evaluation of all 5 models, ResNet18 is selected as the

best-performing model. The training curve of ResNet152 is shown in Figure 5.

4.2 Fine-tuning of ResNet18

The best-performing model of 4, the ResNet18 was further evaluated to find the best learning rate. The loss vs. learning rate plot for the test set and validation set are shown in 6. For a learning rate of 2.2×10^{-6} , the training and validation loss was lowest. Here, it decreases with the course of the learning rate, showing that the model is learning well. But after a threshold value is crossed—roughly 10^{-6} , the loss starts to increase, reflecting unsatisfactory learning behavior. This is indicative of the model sensitivity by the chosen learning rate and emphasizes important selection so that convergence is ensured.

After selecting the learning rate the previously trained ResNet18 model was further trained by more than 50 epochs as shown in

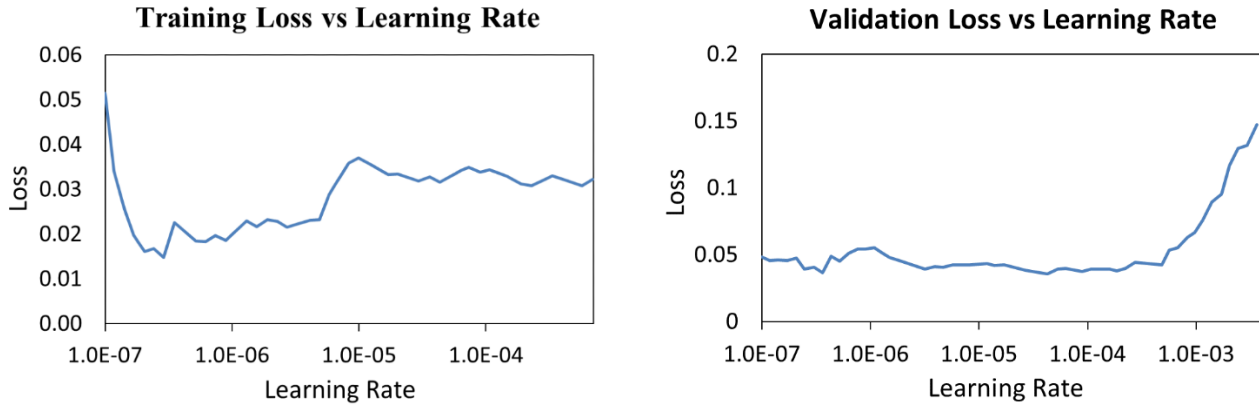


Figure 6: Training Loss vs learning rate and Validation Loss vs learning rate for the ResNet18 model

Confusion matrix

Actual \ Predicted	back	ipsweep	neptune	nmap	normal	pod	portsweep	satan	smurf	teardrop	warezclient
back	6	0	0	0	0	0	0	0	0	0	0
ipsweep	0	15	0	0	0	0	0	0	0	0	0
neptune	0	0	224	0	0	0	0	0	0	0	0
nmap	0	5	0	1	0	0	0	0	0	0	0
normal	0	1	0	0	327	0	0	0	0	0	0
pod	0	0	0	0	0	1	0	0	0	0	0
portsweep	0	0	0	0	0	0	19	0	0	0	0
satan	0	0	0	0	0	0	0	13	0	0	0
smurf	0	0	0	0	0	0	0	1	20	0	0
teardrop	0	0	0	0	0	0	0	0	0	5	0
warezclient	0	0	0	0	0	0	0	0	0	0	4

Figure 7: Confusion matrix for the various classes of attacks

Figure 5. However, after 4 epochs, the training loss decreased gradually but the validation loss increased. It indicates overfitting. So, weights of the model parameters of the 4th epochs are the weights contributing to the best performance of the model.

4.3 Confusion matrix

For a thorough evaluation, a confusion matrix was generated for the ResNet18 model. As shown in Figure 7 below, the confusion matrix investigates model classification performance for various classes.

The diagonal elements refer to successful predictions, whereas non-diagonal elements reflect misclassifications. It is seen that the model predicts all the classes except 'nmap' and 'normal' with 100% accuracy. Some misclassifications can be spotted, mainly for the

class 'nmap', which was wrongly classified as 'normal' five times, and 'normal' was wrongly classified 1 time.

These misclassifications can primarily be attributed to the overlapping characteristics between certain network traffic features in these classes. For instance, 'nmap' is often used in network mapping and reconnaissance, which may exhibit behavioral similarities to benign traffic when observed superficially. This resemblance might cause the model to misclassify 'nmap' activity as 'normal,' particularly when the nuances distinguishing it from legitimate behavior are subtle. There may be added contribution from the color-mapping approach used in order to transform the NSL-KDD dataset into an image dataset. While CNNs detect spatial patterns in image data with great efficiency, some network traffic patterns may not be differentiable enough by their color-mapped form. Subtle similarities of RGB-encoded features across these classes may bring confusion and not allow the model to correctly tell them apart.

4.4 Precision, Recall and f1 score

The precision, recall and f1 score for the model is given in 5. It shows that 6 classes have a f1 score of 1. While the f1 score of nmap is very low (0.2857).

The precision of normal class is 1 meaning FP for normal class is zero. In other words, no attacks were classified as normal connection in this model. Which is very necessary for an IDS.

4.5 Comparison with some other models

A comparison with other models trained on NSL-KDD dataset is in the 6 below. The accuracy of our model (98.91%) was better than these models. Only one model had better f1 score than the model of this paper.

5 CONCLUSION

The research justifies the effectiveness of employing pre-trained weights from CNN for the detection of intrusions, utilizing the NSL-KDD dataset as a foundational benchmark. By converting the dataset rows into image representations and implementing a variety of deep learning architectures—including ResNet18, ResNet50, and BEiT2_base_patch16_224—this study has identified ResNet18 as

Table 5: Precision, recall and f1 score of each classes

Class	Precision	Recall	F1 Score
back	1	1	1
ipsweep	0.714286	1	0.833333
neptune	1	1	1
nmap	1	0.166667	0.285714
normal	1	0.996951	0.998473
pod	1	1	1
portsweep	1	1	1
satan	0.928571	1	0.962963
smurf	1	0.952381	0.97561
teardrop	1	1	1
warezclient	1	1	1
Macro Average	0.967532	0.919636	0.91419

Table 6: Validation metrics of various model trained on NSL-KDD dataset

Model	Accuracy	F1 score	Reference
LSTM	83.68	82.76	[19]
GRU	82.87	83.05	[19]
BLS	84.15	84.68	[19]
Bi-LSTM	81.03	81.23	[19]
RF	80.67	-	[33]
SVM	69.32	-	[33]
RT	81.59	-	[33]
MP	77.41	-	[33]
BC + KNN	94.92	95.39	[18]

the most effective model, achieving a good accuracy rate of 98.91%. From the confusion matrix, it was clear that the model can detect all intrusion as intrusion with no false negative for the validation set. This demonstrates the practical applicability of this model in IDSs. The thorough analysis performed over diverse architectures complemented by fine-tuning illustrates the ease with which transfer learning can be adapted to small-sized data as well, without compromising its accuracy or efficiency.

The results throw light on the substantial benefits of deep learning model integration, especially in those scenarios with high order feature dimensional space and a relatively low dataset size. This fact is supported by the excellent performance of ResNet18 which has a simple architecture, lower losses in training and validation, and requires less time in training. In the validation dataset, one normal connection was detected as an intrusion for only one time with a false positive rate (FPR) of 3.05%. However, the 0% false negative rate (FNR) of this model demonstrates the potential of this model for Anomaly detection in IDSs.

However, there are still some shortcomings in this model for classifying certain types of attacks, such as 'nmap'. Hence, there is room for improvement. Future work can be done on more sophisticated and hybrid datasets. This can improve the attack classification performance of this model even for very rare and modern attack types.

Acknowledgments

We would like to thank the developers of the NSL-KDD dataset, whose elaborate dataset has served as an important asset for strengthening intrusion detection system research. Its well-chosen features helped us much in designing and training our deep learning models. Additionally, we are grateful for the availability of pre-trained weights used in various deep learning architectures since this sped up significantly in retraining and saving on more cycles of fine-tuning a model. This has been invaluable in contributing to the success of this research.

References

- [1] Preeti Aggarwal and Sudhir Kumar Sharma. 2015. Analysis of KDD Dataset Attributes - Class wise for Intrusion Detection. *Procedia Computer Science* 57, (January 2015), 842–851. <https://doi.org/10.1016/j.procs.2015.07.490>
- [2] Akashdeep, Ishfaq Manzoor, and Neeraj Kumar. 2017. A feature reduced intrusion detection system using ANN classifier. *Expert Systems with Applications* 88, (December 2017), 249–257. <https://doi.org/10.1016/j.eswa.2017.07.005>
- [3] Nirjhor Anjum and Md Rubel Chowdhury. 2024. International Journal of Advanced Research in Computer and Communication Engineering. *SSRN Journal* (2024). <https://doi.org/10.2139/ssrn.4847308>
- [4] Nirjhor Anjum and Md Rubel Chowdhury. 2024. A Study on NSL-KDD Dataset for Intrusion Detection System Based on Classification Algorithms. *SSRN Journal* (2024). <https://doi.org/10.2139/ssrn.4847308>
- [5] Rebecca Bace and Peter Mell. 2001. Intrusion Detection Systems. *NIST* (November 2001). Retrieved August 25, 2024 from <https://www.nist.gov/publications/intrusion-detection-systems>
- [6] Siva Surya Narayana Chintapalli, Satya Prakash Singh, Jaroslav Frnda, Parameshchhari Bidare Divakarachari, Vijaya Lakshmi Sarraju, and Przemysław

- Falkowski-Gilski. 2024. OOA-modified Bi-LSTM network: An effective intrusion detection framework for IoT systems. *Heliyon* 10, 8 (April 2024), e29410. <https://doi.org/10.1016/j.heliyon.2024.e29410>
- [7] Allard Dijk, Emre Halisdemir, Cosimo Melella, Alari Schu, Mauno Pihelgas, and Roland Meier. 2024. LSPR23: A novel IDS dataset from the largest live-fire cybersecurity exercise. *Journal of Information Security and Applications* 85, (September 2024), 103847. <https://doi.org/10.1016/j.jisa.2024.103847>
- [8] Amir Djenna, Saad Harous, and Djamel Eddine Saidouni. 2021. Internet of Things Meet Internet of Threats: New Concern Cyber Security Issues of Critical Cyber Infrastructure. *Applied Sciences* 11, 10 (January 2021), 4580. <https://doi.org/10.3390/app11104580>
- [9] Hadeel Q. Ghani and Wathiq L. Al-Yaseen. 2024. Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset. *e-Prime - Advances in Electrical Engineering, Electronics and Energy* 9, (September 2024), 100673. <https://doi.org/10.1016/j.prime.2024.100673>
- [10] Abdul Hamid, Monsur Alam, Hafsina Sheherin, and Al-Sakib Khan Pathan. 2022. Cyber Security Concerns in Social Networking Service. *Int. j. commun. netw. inf. secur.* 12, 2 (April 2022). <https://doi.org/10.17762/ijcnis.v12i2.4634>
- [11] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016. IEEE, Las Vegas, NV, USA, 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [12] Soumyadeep Hore, Jalal Ghadermazi, Ankit Shah, and Nathaniel D. Bastian. 2024. A sequential deep learning framework for a robust and resilient network intrusion detection system. *Computers & Security* 144, (September 2024), 103928. <https://doi.org/10.1016/j.cose.2024.103928>
- [13] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. 2016. Deep Networks with Stochastic Depth. In *Computer Vision – ECCV 2016*, 2016. Springer International Publishing, Cham, 646–661. https://doi.org/10.1007/978-3-319-46493-0_39
- [14] Mohammed Ishaque, Md Gapar Md Johar, Ali Khatibi, and Muhammed Yamin. 2023. A novel hybrid technique using fuzzy logic, neural networks and genetic algorithm for intrusion detection system. *Measurement: Sensors* 30, (December 2023), 100933. <https://doi.org/10.1016/j.measen.2023.100933>
- [15] Danish Javed, Muhammad Shahid Saeed, Muhammad Adil, Prabhat Kumar, and Alireza Jolfaei. 2024. A federated learning-based zero trust intrusion detection system for Internet of Things. *Ad Hoc Networks* 162, (September 2024), 103540. <https://doi.org/10.1016/j.adhoc.2024.103540>
- [16] Dai Jianjian, Tao Yang, and Yang Feiyue. 2018. A Novel Intrusion Detection System based on IABRF SVM for Wireless Sensor Networks. *Procedia Computer Science* 131, (January 2018), 1113–1121. <https://doi.org/10.1016/j.procs.2018.04.275>
- [17] RICH LEE and ING-YI CHEN. 2020. The Time Complexity Analysis of Neural Network Model Configurations. In *2020 International Conference on Mathematics and Computers in Science and Engineering (MACISE)*, January 2020. 178–183. <https://doi.org/10.1109/MACISE49704.2020.00039>
- [18] Longjie Li, Yang Yu, Shenshen Bai, Ying Hou, and Xiaoyun Chen. 2018. An Effective Two-Step Intrusion Detection Approach Based on Binary Classification and k-NN. *IEEE Access* 6, (2018), 12060–12073. <https://doi.org/10.1109/ACCESS.2017.2787719>
- [19] Zhida Li, Prerna Batta, and Ljiljana Trajkovic. 2018. Comparison of Machine Learning Algorithms for Detection of Network Intrusions. In *2018 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, October 2018. IEEE, Miyazaki, Japan, 4248–4253. <https://doi.org/10.1109/SMC.2018.00719>
- [20] Junwei Liang, Muhammad Sadiq, Geng Yang, Kai Jiang, Tie Cai, and Maode Ma. 2024. Enhanced collaborative intrusion detection for industrial cyber-physical systems using permissioned blockchain and decentralized federated learning networks. *Engineering Applications of Artificial Intelligence* 135, (September 2024), 108862. <https://doi.org/10.1016/j.engappai.2024.108862>
- [21] Hung-Jen Liao, Chun-Hung Richard Lin, Ying-Chih Lin, and Kuang-Yuan Tung. 2013. Intrusion detection system: A comprehensive review. *Journal of Network and Computer Applications* 36, 1 (January 2013), 16–24. <https://doi.org/10.1016/j.jnca.2012.09.004>
- [22] Luigi F. Marques da Luz, Paulo Freitas de Araujo-Filho, and Divanilson R. Campelo. 2024. Multi-stage deep learning-based intrusion detection system for automotive Ethernet networks. *Ad Hoc Networks* 162, (September 2024), 103548. <https://doi.org/10.1016/j.adhoc.2024.103548>
- [23] Zhiyu Ma, Chen Li, Tianming Du, Le Zhang, Dechao Tang, Deguo Ma, Shanchuan Huang, Yan Liu, Yihao Sun, Zhihao Chen, Jin Yuan, Qianqing Nie, Marcin Grzegorz, and Hongzan Sun. 2024. AATCT-IDS: A benchmark Abdominal Adipose Tissue CT Image Dataset for image denoising, semantic segmentation, and radiomics evaluation. *Computers in Biology and Medicine* 177, (July 2024), 108628. <https://doi.org/10.1016/j.combiomed.2024.108628>
- [24] Ehsan Mahdavi, Ali Fanian, Abdolreza Mirzaei, and Zahra Taghiyarrenani. 2022. ITL-IDS: Incremental Transfer Learning for Intrusion Detection Systems. *Knowledge-Based Systems* 253, (October 2022), 109542. <https://doi.org/10.1016/j.knsys.2022.109542>
- [25] Ashfaq Ahmad Najjar and Manohar Naik S. 2024. A Robust DDoS Intrusion Detection System Using Convolutional Neural Network. *Computers and Electrical Engineering* 117, (July 2024), 109277. <https://doi.org/10.1016/j.compeleceng.2024.109277>
- [26] Babatunde Olanrewaju-George and Bernardi Pranggono. 2025. Federated learning-based intrusion detection system for the internet of things using unsupervised and supervised deep learning models. *Cyber Security and Applications* 3, (December 2025), 100068. <https://doi.org/10.1016/j.csa.2024.100068>
- [27] Vijay Paidi, Hasan Fleyeh, and Roger G. Nyberg. 2020. Deep learning-based vehicle occupancy detection in an open parking lot using thermal camera. *IET Intelligent Transport Systems* 14, 10 (2020), 1295–1302. <https://doi.org/10.1049/iet-its.2019.0468>
- [28] Benedetto Marco Serinelli, Anastasija Collen, and Niels Alexander Nijdam. 2020. Training Guidance with KDD Cup 1999 and NSL-KDD Data Sets of ANIDINR: Anomaly-Based Network Intrusion Detection System. *Procedia Computer Science* 175, (January 2020), 560–565. <https://doi.org/10.1016/j.procs.2020.07.080>
- [29] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization: In *Proceedings of the 4th International Conference on Information Systems Security and Privacy*, 2018. SCITEPRESS - Science and Technology Publications, Funchal, Madeira, Portugal, 108–116. <https://doi.org/10.5220/0006639801080116>
- [30] Jiaru Song, Guihe Qin, Yanhua Liang, Jie Yan, and Minghui Sun. 2024. SIDI-DNG: A similarity-based intrusion detection system using improved Levenshtein Distance and N-gram for CAN. *Computers & Security* 142, (July 2024), 103847. <https://doi.org/10.1016/j.cose.2024.103847>
- [31] Jiaru Song, Guihe Qin, Yanhua Liang, Jie Yan, and Minghui Sun. 2024. DGIDS: Dynamic graph-based intrusion detection system for CAN. *Computers & Security* (August 2024), 104076. <https://doi.org/10.1016/j.cose.2024.104076>
- [32] Peter Stavroulakis and Mark Stamp. 2010. *Handbook of Information and Communication Security*. <https://doi.org/10.1007/978-3-642-04117-4>
- [33] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali Ghorbani. 2009. A detailed analysis of the KDD CUP 99 data set. *IEEE Symposium. Computational Intelligence for Security and Defense Applications, CISDA 2*, (July 2009). <https://doi.org/10.1109/CISDA.2009.5356528>
- [34] Ankit Thakkar and Ritika Lohiya. 2020. A Review of the Advancement in Intrusion Detection Datasets. *Procedia Computer Science* 167, (January 2020), 636–645. <https://doi.org/10.1016/j.procs.2020.03.330>
- [35] Fadila Tili, Samiha Ayed, and Lamia Chaari Fourati. 2024. Exhaustive distributed intrusion detection system for UAVs attacks detection and security enforcement (E-DIDS). *Computers & Security* 142, (July 2024), 103878. <https://doi.org/10.1016/j.cose.2024.103878>
- [36] Amol D. Vibhute, Chandrashekar H. Patil, Arjun V. Mane, and Karbhari V. Kale. 2024. Towards Detection of Network Anomalies using Machine Learning Algorithms on the NSL-KDD Benchmark Datasets. *Procedia Computer Science* 233, (January 2024), 960–969. <https://doi.org/10.1016/j.procs.2024.03.285>
- [37] Shun-Sheng Wang, Kuo-Qin Yan, Shu-Ching Wang, and Chia-Wei Liu. 2011. An Integrated Intrusion Detection System for Cluster-based Wireless Sensor Networks. *Expert Systems with Applications* 38, 12 (November 2011), 15234–15243. <https://doi.org/10.1016/j.eswa.2011.05.076>
- [38] Wufei Wu and Javad Hassannataj Joloudari. 2024. Deep Transfer Learning Techniques in Intrusion Detection System-Internet of Vehicles: A State-of-the-Art Review. *Computers, Materials and Continua* 80, 2 (August 2024), 2785–2813. <https://doi.org/10.32604/cmc.2024.053037>
- [39] Yan Zhang, Degang Zhu, Menglin Wang, Junhan Li, and Jie Zhang. 2024. A comparative study of cyber security intrusion detection in healthcare systems. *International Journal of Critical Infrastructure Protection* 44, (March 2024), 100658. <https://doi.org/10.1016/j.ijcip.2023.100658>
- [40] Umer Zukaib, Xiaohui Cui, Chengliang Zheng, Dong Liang, and Salah Ud Din. 2024. Meta-Fed IDS: Meta-learning and Federated learning based fog-cloud approach to detect known and zero-day cyber attacks in IoMT networks. *Journal of Parallel and Distributed Computing* 192, (October 2024), 104934. <https://doi.org/10.1016/j.jpdc.2024.104934>
- [41] 2024. CNN Channel Attention Intrusion Detection System Using NSL-KDD Dataset. *Computers, Materials and Continua* 79, 3 (June 2024), 4319–4347. <https://doi.org/10.32604/cmc.2024.050586>
- [42] NSL-KDD. Retrieved August 25, 2024 from <https://www.kaggle.com/datasets/hassan06/nslkdd>